ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		В.В. Жукалин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 2

Создание структуры веб-приложения с помощью App Router. Проектирование авторизации

по курсу: Web-программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321		Г.В. Буренков
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Продолжение темы PairWise 4
3 Маршруты приложения 5
4 Реализация 7
4.1 Страницы и корневой макет 7
4.2 Динамическая карточка площадки 7
4.3 Учебный вход и кабинет 8
5 Результаты 10
6 Ответы на контрольные вопросы 14
ЗАКЛЮЧЕНИЕ 16
ПРИЛОЖЕНИЕ А Листинги исходного кода 17

1 Цель работы

Цель работы — продолжить приложение PairWise и собрать его структуру на App Router: общие страницы, корневой макет, динамическую карточку площадки и учебные формы входа и регистрации.

2 Продолжение темы PairWise

PairWise подбирает коворкинг для очной встречи. Пользователь смотрит площадки по городу, адресу и стеку людей, которые там бывают. Главная сущность каталога — площадка. У неё есть номер, название, описание, город, адрес, список технологий, изображение и дата появления в базе.

В учебном массиве шесть площадок Санкт-Петербурга: «Точка Кипения», GrowUp Ligovsky, Bench на Петроградке, Севкабель Порт, ITMO Cowork и «Практик на Васильевском». Номера идут от 0 до 5. Каталог показывает карточки этих площадок, а страница /catalog/[id] открывает одну из них.

3 Маршруты приложения

Адрес страницы собирается из вложенных каталогов app, в которых лежит page.tsx. Каталоги data, providers и ui адреса не создают: в них данные, состояние и компоненты. Корневой layout.tsx задаёт язык ru, шрифт Manrope, шапку и оборачивает все страницы в DemoAuthProvider.

Таблица 1 — Назначение раздела, каталог app и URL

Назначение	**Каталог app**	**URL**
Главная	app/page.tsx	/
Переход на главную	app/start/page.tsx	/start
О проекте	app/about/page.tsx	/about
Каталог площадок	app/catalog/page.tsx	/catalog
Карточка площадки	app/catalog/[id]/page.tsx	/catalog/[id]
Вход	app/auth/login/page.tsx	/auth/login
Регистрация	app/auth/register/page.tsx	/auth/register
Личный кабинет	app/cabinet/page.tsx	/cabinet
Неизвестный адрес	app/not-found.tsx	notFound()
Общий макет	app/layout.tsx	для всех URL

Страница /start выполняет redirect на /. Меню шапки собрано через Link. Текущий пункт определяется хуком usePathname и помечается атрибутом aria-current. В меню также есть «Карта» (/map) и «Избранное» (/saved). Отдельных page.tsx для этих адресов нет, поэтому открывается not-found.

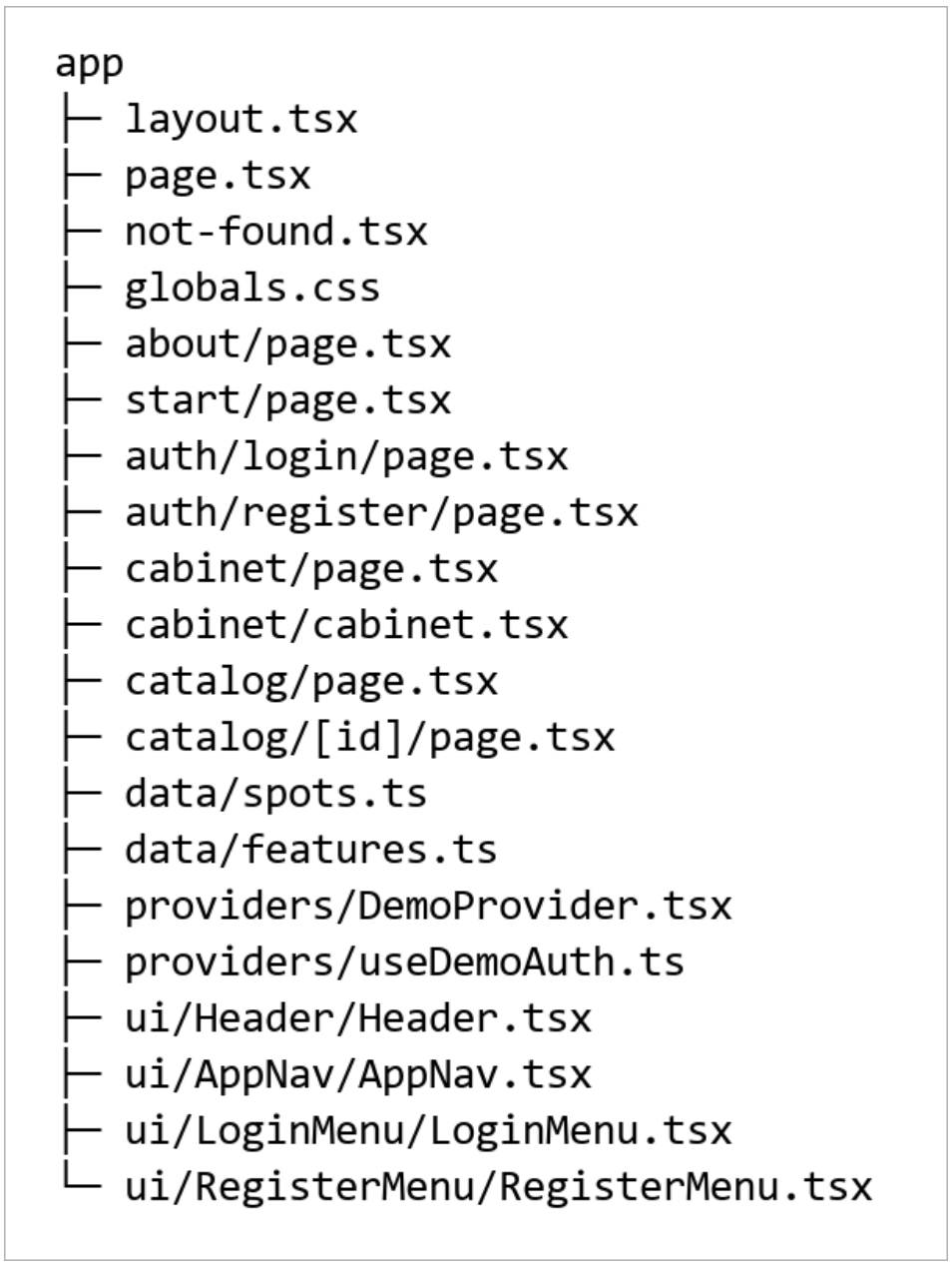


Рисунок 1 — Структура каталога app

4 Реализация

4.1 Страницы и корневой макет

Файл page.tsx задаёт содержимое одного адреса. Файл layout.tsx адрес не добавляет: он рисует общую оболочку и остаётся на экране при переходе между вложенными страницами. Корневой макет один на всё приложение. В нём подключены шапка, кнопка прокрутки наверх и провайдер учебного пользователя.

Страницы главной, каталога, карточки, проекта, входа, регистрации и кабинета серверные: директивы «use client» в их page.tsx нет. Страница входа только выводит LoginMenu, страница регистрации — RegisterMenu, страница кабинета — CabinetView. Интерактивные части вынесены в клиентские компоненты. Директива стоит у DemoAuthProvider, AppNav, LoginMenu, RegisterMenu и CabinetView, потому что им нужны состояние, контекст или события формы.

4.2 Динамическая карточка площадки

Каталог app/catalog/[id] создаёт динамический сегмент. Квадратные скобки в имени означают, что на место id в адрес подставляется конкретное значение: /catalog/0, /catalog/1 и дальше. Страница объявлена как async и получает params через await: в текущем App Router это Promise. Строка id переводится в число. Если число не получилось или площадки с таким номером нет, вызывается notFound().

Карточка показывает фотографию, название, описание, город, адрес, дату попадания в базу и стек площадки. Ссылка «К каталогу» возвращает на /catalog. Из списка площадок переход идёт по адресу /catalog/ и номеру карточки. Открыты /catalog/0 («Точка Кипения») и /catalog/1 (GrowUp Ligovsky). Адрес /catalog/999 попадает в not-found: на экране текст «404 ха ха» и ссылка «Вернуться на главную».

4.3 Учебный вход и кабинет

DemoAuthProvider хранит учебного пользователя в useState. В объекте два поля: name и email. Функция login принимает почту и берёт имя из части адреса до символа @. Если такой части нет, имя становится «Пользователь». Функция register записывает переданные имя и почту. logout возвращает состояние к null. Хук useDemoAuth читает контекст и выбрасывает ошибку, если его вызвали вне провайдера.

Пароль в login и register не передаётся и в состоянии не хранится. На форме входа есть флажок «Запомнить логин и пароль», но обработчик отправки его не читает. Подсказка поля пароля говорит «Не менее 8 символов». Длину форма входа не проверяет: пустое поле останавливает атрибут required. На регистрации сравниваются два введённых пароля. Если они различаются, под полями появляется текст «Пароли не совпадают», и переход в кабинет не выполняется. Дата рождения на форме есть, в провайдер она не уходит.

Кабинет без пользователя показывает заголовок, фразу «Войдите или зарегистрируйтесь, чтобы открыть кабинет» и ссылки на вход и регистрацию. После учебного входа на странице приветствие с именем, почта и кнопка «Выйти». Шапка состояние не читает: ссылка «Вход» на всех экранах ведёт на /auth/login. Имя появляется в кабинете.

Таблица 2 — Аутентификация, сессия и авторизация

Понятие	**Назначение**	**Как сделано в PairWise**
Аутентификация	Установить, кто обращается к системе	Форма принимает почту. Пароль ни с чем не сравнивается и в состояние не записывается
Сессия	Сохранить результат входа между запросами	Имя и почта лежат в памяти вкладки. Обновление страницы возвращает пустое состояние
Авторизация	Решить, какое действие разрешено	Проверки прав нет. Кабинет меняет текст, если пользователь уже записан

Прототип показывает сценарий гостя и сценарий учебного пользователя. Отдельной надписи о том, что вход учебный, на формах нет. Подтверждения личности и выдачи прав в этом состоянии нет. Состояние создаётся заново при загрузке дерева React, в cookie и localStorage провайдер ничего не пишет. Условие «пользователь есть» меняет только разметку кабинета. Скрытая кнопка на странице не закрывает серверную операцию: ту же операцию можно вызвать запросом мимо интерфейса. Базы пользователей и серверной сессии в работе нет. После подключения PostgreSQL и Prisma на их место встанут учётная запись, хеш пароля, серверная сессия и проверка права перед действием. Формы можно оставить, отправлять их нужно будет уже на сервер.

5 Результаты

Ниже экраны работающего приложения. В package.json для статической проверки и производственной сборки заданы команды npm run lint и npm run build.

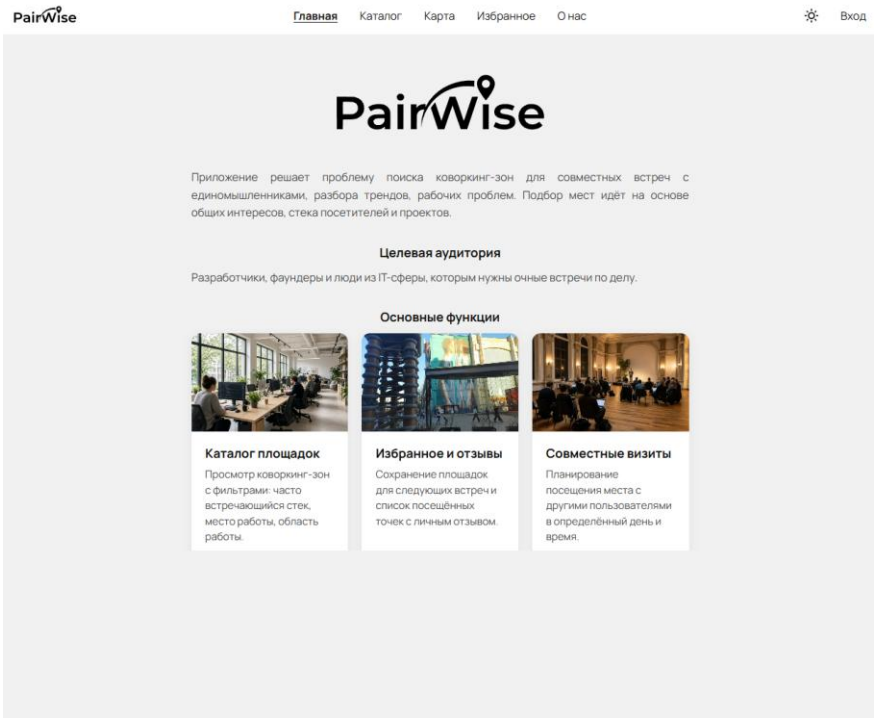


Рисунок 2 — Главная страница PairWise

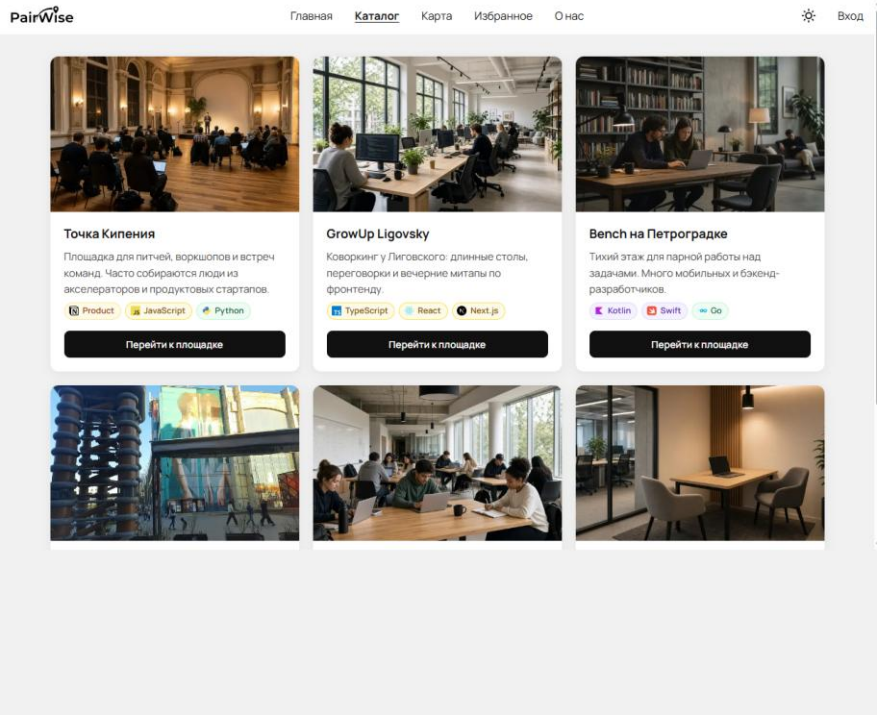


Рисунок 3 — Каталог площадок

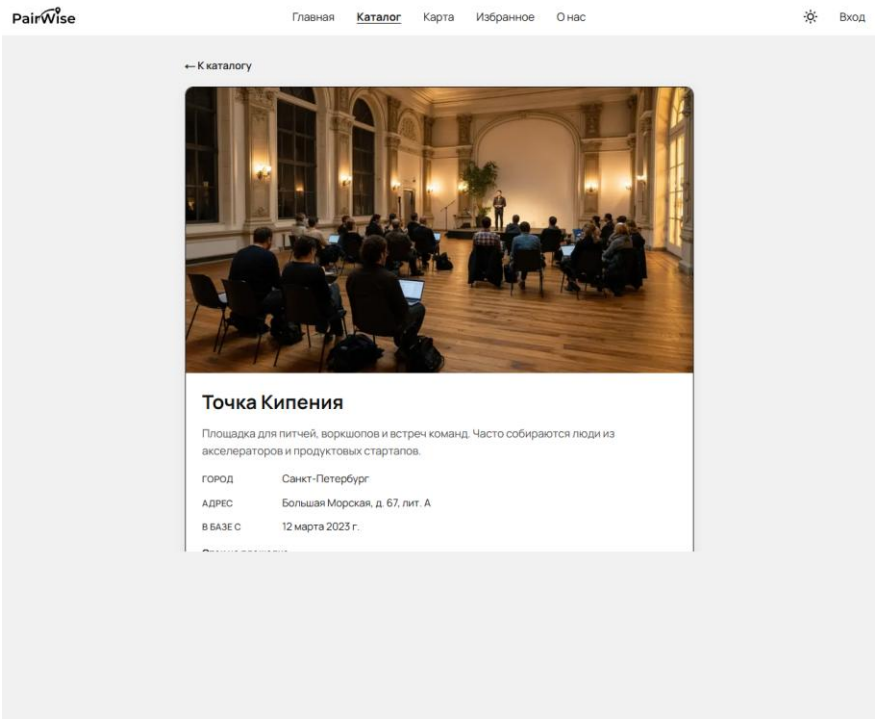


Рисунок 4 — Карточка «Точка Кипения», адрес /catalog/0

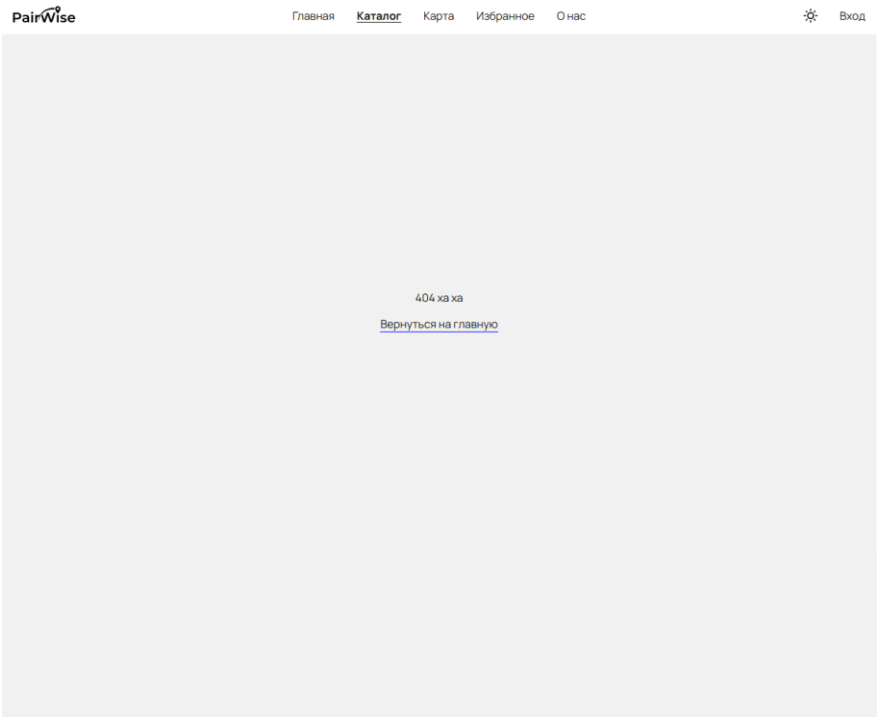


Рисунок 5 — Неизвестный идентификатор /catalog/999

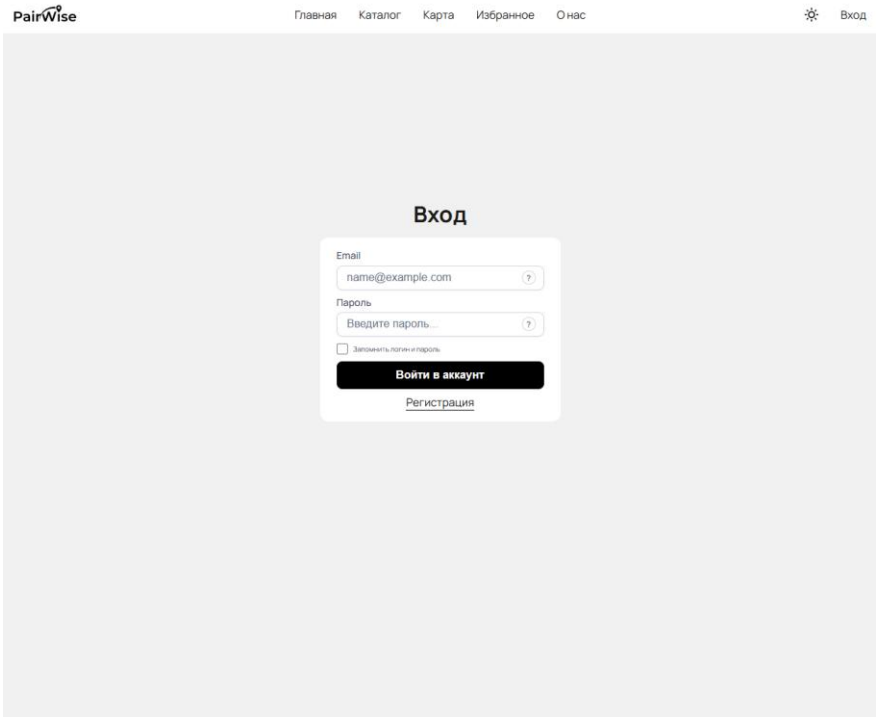


Рисунок 6 — Форма входа

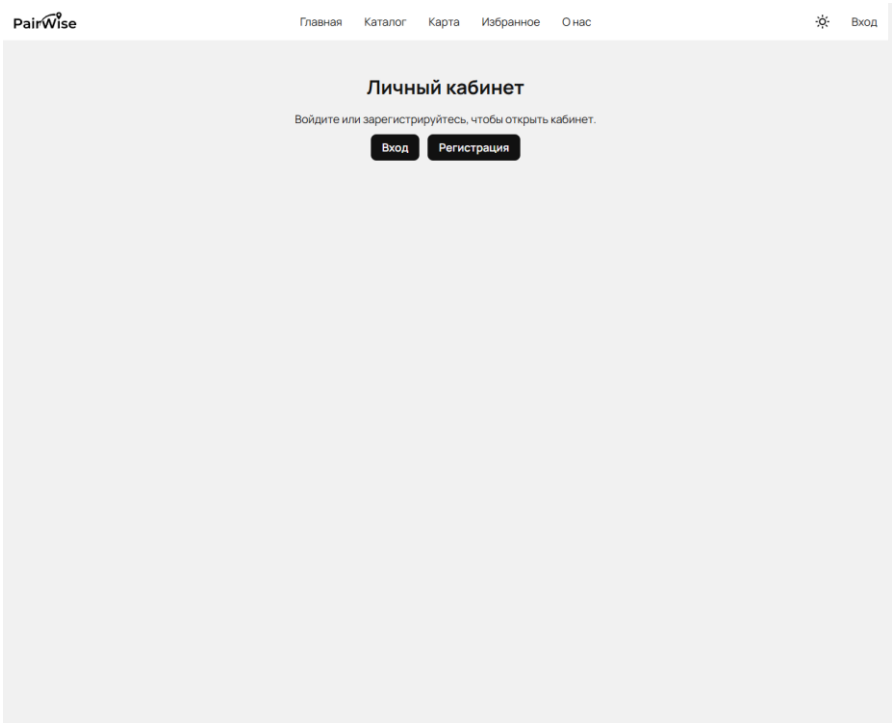


Рисунок 7 — Личный кабинет до входа

На главной виден текст о подборе коворкинга и блок функций. В каталоге карточки площадок со стеком и ссылкой «Перейти к площадке». На карточке /catalog/0 показаны город, адрес и дата. Для номера 999 открывается страница неизвестного адреса. Форма входа содержит почту, пароль, флажок и ссылку на регистрацию. Кабинет до входа предлагает войти или зарегистрироваться. В шапке на этих экранах остаётся пункт «Вход».

6 Ответы на контрольные вопросы

Как каталог app преобразуется в URL?

Вложенный каталог с файлом page.tsx добавляет сегмент адреса. Файл app/catalog/page.tsx открывается как /catalog, а app/catalog/[id]/page.tsx — как /catalog/ и значение сегмента. Каталог без page.tsx, например data или ui, адресом не становится.

Чем page.tsx отличается от layout.tsx?

page.tsx — содержимое конкретного адреса. layout.tsx — оболочка вокруг этого содержимого и вложенных страниц. При переходе по Link оболочка сохраняется, меняется страница внутри неё.

Для чего используются квадратные скобки в имени каталога?

Скобки обозначают динамический сегмент. Имя внутри скобок, здесь id, становится полем params. Один файл страницы обслуживает все карточки площадок.

Почему params в актуальном App Router получают через await?

В текущей версии params передаётся как Promise. Страница объявлена async и сначала дожидается этого значения, затем читает id.

Когда требуется директива «use client»?

Она нужна компоненту с состоянием, контекстом, обработчиком события или хуком браузера. В PairWise это провайдер, меню, формы и кабинет. Серверная страница может подключить такой компонент и сама остаться серверной.

Какие преимущества даёт Link по сравнению с полной перезагрузкой?

Link меняет адрес без полной перезагрузки документа. Общий макет и дерево React сохраняются, поэтому учебный пользователь остаётся в памяти,

пока вкладку не обновят. Обычный переход с перезагрузкой собирает страницу заново.

Чем аутентификация отличается от авторизации?

Аутентификация устанавливает, кто обращается. Авторизация решает, что этому человеку разрешено. Форма PairWise записывает почту в состояние и права на действия не проверяет.

Для чего нужна сессия?

Сессия хранит уже установленный результат входа между запросами, чтобы пользователь не представлялся на каждой странице заново. В прототипе эту роль играет useState внутри вкладки.

Почему скрытая кнопка не защищает серверную операцию?

Кнопка и условие в разметке работают в браузере. Серверная операция обязана проверять право сама. Если проверки на сервере нет, запрос можно отправить мимо спрятанной кнопки.

Почему DemoAuthProvider сбрасывается после обновления страницы?

Состояние создаётся в памяти вкладки при монтировании провайдера. Начальное значение user равно null. Обновление страницы монтирует дерево заново. В хранилище браузера и в cookie провайдер запись не делает.

Какие части прототипа будут заменены после подключения PostgreSQL и Prisma?

Вместо объекта в useState появятся запись пользователя и хеш пароля в базе, серверная проверка пароля, сессия между запросами и проверка права перед действием. Клиентские формы могут остаться, но отправка уйдёт на сервер.

ЗАКЛЮЧЕНИЕ

В работе продолжен проект PairWise. На App Router собраны главная страница, страница о проекте, каталог, динамическая карточка площадки, вход, регистрация и личный кабинет. Общий макет держит шапку и провайдер учебного пользователя. Неизвестный номер площадки обрабатывается через notFound().

Учебный вход хранит имя и почту в памяти вкладки. Пароль в это состояние не попадает. После обновления страницы записанный пользователь пропадает. Такой прототип показывает сценарий интерфейса и не заменяет проверку личности, сессию и права на сервере.

ПРИЛОЖЕНИЕ А Листинги исходного кода

Листинг А.1 — Файл app/data/spots.ts

```
export type SpotType =
    {
        id: number,
        name: string,
        description: string,
        city: string,
        adress: string,
        stack: string[],
        image: string,
        coords: {x: number, y: number},
        createdAt: Date,
    }

export const spots: SpotType[] = [
    {
        id: 0,
        name: "Точка Кипения",
        description: "Площадка для питчей, воркшопов и встреч команд. Часто
собираются люди из акселераторов и продуктовых стартапов.",
        city: "Санкт-Петербург",
        adress: "Большая Морская, д. 67, лит. А",
        stack: ["Product", "JavaScript", "Python"],
        image: "/images/spots/tochka-kipeniya.jpg",
        coords: { x: 30.2884, y: 59.9318 },
        createdAt: new Date("2023-03-12"),
    },
    {
        id: 1,
        name: "GrowUp Ligovsky",
        description: "Коворкинг у Лиговского: длинные столы, переговорки и
вечерние митапы по фронтенду.",
        city: "Санкт-Петербург",
        adress: "Лиговский проспект, д. 50, корп. 12",
        stack: ["TypeScript", "React", "Next.js"],
        image: "/images/spots/growup-ligovsky.jpg",
        coords: { x: 30.3612, y: 59.9274 },
        createdAt: new Date("2023-06-01"),
    },
    {
        id: 2,
        name: "Bench на Петроградке",
        description: "Тихий этаж для парной работы над задачами. Много
мобильных и бэкенд-разработчиков.",
        city: "Санкт-Петербург",
        adress: "Каменноостровский проспект, д. 10",
        stack: ["Kotlin", "Swift", "Go"],
        image: "/images/spots/bench-petrogradka.jpg",
        coords: { x: 30.3148, y: 59.9601 },
        createdAt: new Date("2023-09-18"),
    },
    {
        id: 3,
        name: "Севкабель Порт, коворкинг",
        description: "Открытое пространство у воды. Подходит, чтобы разобрать
тренды и показать демо с ноутбука.",
        city: "Санкт-Петербург",
        adress: "Кожевенная линия, д. 40",
```

```
        stack: ["UX", "Figma", "React"],
        image: "/images/spots/sevkabel.jpg",
        coords: { x: 30.2419, y: 59.9246 },
        createdAt: new Date("2024-01-09"),
    },
    {
        id: 4,
        name: "ITMO Cowork",
        description: "Кампусная зона возле ИТМО: студенты, ML-кружки и
вечерние разборы пет-проектов.",
        city: "Санкт-Петербург",
        adress: "Кронверкский проспект, д. 49",
        stack: ["Python", "ML", "C++"],
        image: "/images/spots/itmo-cowork.jpg",
        coords: { x: 30.3101, y: 59.9571 },
        createdAt: new Date("2024-04-22"),
    },
    {
        id: 5,
        name: "Практик на Васильевском",
        description: "Небольшой коворкинг с переговоркой на двоих. Удобно
сесть с человеком из той же области работы.",
        city: "Санкт-Петербург",
        adress: "Средний проспект В.О., д. 36/40",
        stack: ["Java", "Spring", "DevOps"],
        image: "/images/spots/praktik-vo.jpg",
        coords: { x: 30.2776, y: 59.9423 },
        createdAt: new Date("2024-08-05"),
    },
]
```

Листинг А.2 — Файл app/catalog/page.tsx

```
import { spots } from "../data/spots";
import SpotCard from "../ui/SpotCard/SpotCard";
import styles from "./SpotPage.module.css";

const CatalogPage = () => {
    return (
        <main className={styles.catalog}>
            <ul className={styles.catalog__list}>
                {spots.map((spot) => (
                    <li
                        key={spot.id}
                        className={styles.catalog__item}
                    >
                        <SpotCard spot={spot} />
                    </li>
                ))}
            </ul>
        </main>
    );
};

export default CatalogPage;
```

Листинг А.3 — Файл app/catalog/[id]/page.tsx

```
import Image from "next/image";
import Link from "next/link";
import { notFound } from "next/navigation";
import { spots } from "../../data/spots";
import SkillTags from "../../ui/SkillTags/SkillTags";
```

```
import styles from "../SpotPage.module.css";

const SpotPage = async ({
    params,
}: {
    params: Promise<{ id: string }>;
}) => {
    const { id: idParam } = await params;
    const id = Number(idParam);

    if (Number.isNaN(id)) {
        notFound();
    }

    const spot = spots.find((item) => item.id === id);

    if (!spot) {
        notFound();
    }

    const createdLabel = spot.createdAt.toLocaleDateString("ru-RU", {
        day: "numeric",
        month: "long",
        year: "numeric",
    });

    return (
        <main className={styles.spot}>
            <Link
                href="/catalog"
                className={styles.spot__back}
            >
                ← К каталогу
            </Link>
            <article className={styles.spot__card}>
                <div className={styles.spot__media}>
                    <Image
                        src={spot.image}
                        alt={spot.name}
                        fill
                        priority
                        className={styles.spot__image}
                        sizes="(max-width: 768px) 100vw, 48rem"
                    />
                </div>
                <div className={styles.spot__body}>
                    <h1 className={styles.spot__title}>{spot.name}</h1>
                    <p
className={styles.spot__description}>{spot.description}</p>
                    <dl className={styles.spot__meta}>
                        <div className={styles.spot__metaRow}>
                            <dt className={styles.spot__metaTerm}>Город</dt>
                            <dd
className={styles.spot__metaValue}>{spot.city}</dd>
                        </div>
                        <div className={styles.spot__metaRow}>
                            <dt className={styles.spot__metaTerm}>Адрес</dt>
                            <dd
className={styles.spot__metaValue}>{spot.adress}</dd>
                        </div>
                        <div className={styles.spot__metaRow}>
                            <dt className={styles.spot__metaTerm}>В базе
c</dt>
```

```
                            <dd
className={styles.spot__metaValue}>{createdLabel}</dd>
                        </div>
                    </dl>
                    <div className={styles.spot__skills}>
                        <h2 className={styles.spot__skillsTitle}>Стек на
площадке</h2>
                        <SkillTags skills={spot.stack} />
                    </div>
                </div>
            </article>
        </main>
    );
};

export default SpotPage;
```

Листинг A.4 — Файл app/providers/DemoProvider.tsx

```
"use client";

import {
    createContext,
    useMemo,
    useState,
    type ReactNode,
} from "react";

export type DemoUser = {
    name: string;
    email: string;
};

export type DemoAuthContextValue = {
    user: DemoUser | null;
    login: (email: string) => void;
    register: (name: string, email: string) => void;
    logout: () => void;
};

export const DemoAuthContext = createContext<DemoAuthContextValue |
null>(null);

type DemoAuthProviderProps = Readonly<{
    children: ReactNode;
}>;

const DemoAuthProvider = ({ children }: DemoAuthProviderProps) => {
    const [user, setUser] = useState<DemoUser | null>(null);

    const value = useMemo<DemoAuthContextValue>(
        () => ({
            user,
            login(email: string) {
                const localPart = email.split("@")[0]?.trim();
                setUser({
                    name: localPart || "Пользователь",
                    email,
                });
            },
            register(name: string, email: string) {
                setUser({ name, email });
```

```
            },
            logout() {
                setUser(null);
            },
        }),
        [user],
    );

    return (
        <DemoAuthContext.Provider value={value}>
            {children}
        </DemoAuthContext.Provider>
    );
};

export default DemoAuthProvider;
```

Листинг А.5 — Файл app/providers/useDemoAuth.ts

```
import { useContext } from "react";
import { DemoAuthContext } from "./DemoProvider";

export function useDemoAuth () {
    const ctx = useContext(DemoAuthContext)
    if (!ctx) throw new Error("useDemoAuth внутри провайдера")
    return ctx
}
```

Листинг А.6 — Файл app/ui/LoginMenu/LoginMenu.tsx

```
"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type SubmitEventHandler } from "react";
import { useDemoAuth } from "../../providers/useDemoAuth";
import Button from "../Button/Button";
import Input from "../Input/Input";
import styles from "./LoginMenu.module.css";

const LoginMenu = () => {
    const router = useRouter();
    const { login } = useDemoAuth();
    const [email, setEmail] = useState("");

    const handleSubmit: SubmitEventHandler<HTMLFormElement> = (event) => {
        event.preventDefault();
        login(email.trim());
        router.push("/cabinet");
    };

    return (
        <div className={styles.login}>
            <h1 className={styles.login__title}>Вход</h1>
            <form
                className={styles.login__form}
                onSubmit={handleSubmit}
            >
                <div className={styles.login__block}>
                    <Input
                        id="login-email"
                        name="email"
```

```
                        variant="email"
                        className={styles.login__item}
                        label="Email"
                        value={email}
                        onChange={(event) => setEmail(event.target.value)}
                        required
                    />
                    <Input
                        id="login-password"
                        name="password"
                        variant="password"
                        className={styles.login__item}
                        label="Пароль"
                        required
                    />
                    <Input
                        id="login-remember"
                        name="remember"
                        variant="checkbox"
                        className={styles.login__item}
                        label="Запомнить логин и пароль"
                    />
                </div>
                <div className={styles.login__submit}>
                    <Button type="submit">Войти в аккаунт</Button>
                </div>
                <Link
                    href="/auth/register"
                    className={styles.login__link}
                >
                    Регистрация
                </Link>
            </form>
        </div>
    );
};

export default LoginMenu;
```

Листинг А.7 — Файл app/ui/RegisterMenu/RegisterMenu.tsx

```
"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type SubmitEventHandler } from "react";
import { useDemoAuth } from "../../providers/useDemoAuth";
import Button from "../Button/Button";
import Input from "../Input/Input";
import styles from "./RegisterMenu.module.css";

const RegisterMenu = () => {
    const router = useRouter();
    const { register } = useDemoAuth();
    const [firstName, setFirstName] = useState("");
    const [lastName, setLastName] = useState("");
    const [email, setEmail] = useState("");
    const [password, setPassword] = useState("");
    const [passwordRepeat, setPasswordRepeat] = useState("");
    const [passwordError, setPasswordError] = useState("");

    const handleSubmit: SubmitEventHandler<HTMLFormElement> = (event) => {
        event.preventDefault();
```

```
        setPasswordError("");

        if (password !== passwordRepeat) {
            setPasswordError("Пароли не совпадают");
            return;
        }

        const name = `${firstName.trim()} ${lastName.trim()}`.trim();
        register(name, email.trim());
        router.push("/cabinet");
    };

    return (
        <div className={styles.register}>
            <h1 className={styles.register__title}>Регистрация</h1>
            <form
                className={styles.register__form}
                onSubmit={handleSubmit}
            >
                <div className={styles.register__block}>
                    <Input
                        id="register-email"
                        name="email"
                        variant="email"
                        className={styles.register__item}
                        label="Email"
                        placeholder="name@example.com"
                        value={email}
                        onChange={(event) => setEmail(event.target.value)}
                        required
                    />
                    <Input
                        id="register-first-name"
                        name="firstName"
                        variant="text"
                        className={styles.register__item}
                        label="Имя"
                        value={firstName}
                        onChange={(event) =>
setFirstName(event.target.value)}
                        required
                    />
                    <Input
                        id="register-last-name"
                        name="lastName"
                        variant="text"
                        className={styles.register__item}
                        label="Фамилия"
                        value={lastName}
                        onChange={(event) => setLastName(event.target.value)}
                        required
                    />
                    <Input
                        id="register-birthday"
                        name="birthday"
                        variant="date"
                        className={styles.register__item}
                        label="Дата рождения"
                        required
                    />
                    <Input
                        id="register-password"
                        name="password"
```

```
                        variant="password"
                        className={styles.register__item}
                        label="Пароль"
                        value={password}
                        onChange={(event) => setPassword(event.target.value)}
                        required
                    />
                    <Input
                        id="register-password-repeat"
                        name="passwordRepeat"
                        variant="password"
                        className={styles.register__item}
                        label="Повторите пароль"
                        value={passwordRepeat}
                        onChange={(event) =>
setPasswordRepeat(event.target.value)}
                        required
                    />
                    {passwordError && (
                        <p
                            className={styles.register__error}
                            role="alert"
                        >
                            {passwordError}
                        </p>
                    )}
                    <Input
                        id="register-consent"
                        name="consent"
                        variant="checkbox"
                        className={styles.register__item}
                        label="Даю разрешение на обработку персональных
данных"
                        required
                    />
                </div>
                <div className={styles.register__submit}>
                    <Button type="submit">Зарегистрироваться</Button>
                </div>
                <Link
                    href="/auth/login"
                    className={styles.register__link}
                >
                    Вход в аккаунт
                </Link>
            </form>
        </div>
    );
};

export default RegisterMenu;
```

Листинг А.8 — Файл app/cabinet/cabinet.tsx

```
"use client";

import Link from "next/link";
import { useDemoAuth } from "../providers/useDemoAuth";
import styles from "./cabinet.module.css";

const CabinetView = () => {
    const { user, logout } = useDemoAuth();
```

```
    if (!user) {
        return (
            <main className={`${styles.cabinet} container`}>
                <h1 className={styles.cabinet__title}>Личный кабинет</h1>
                <p className={styles.cabinet__text}>
                    Войдите или зарегистрируйтесь, чтобы открыть кабинет.
                </p>
                <div className={styles.cabinet__actions}>
                    <Link
                        href="/auth/login"
                        className={styles.cabinet__link}
                    >
                        Вход
                    </Link>
                    <Link
                        href="/auth/register"
                        className={styles.cabinet__link}
                    >
                        Регистрация
                    </Link>
                </div>
            </main>
        );
    }

    return (
        <main className={`${styles.cabinet} container`}>
            <h1 className={styles.cabinet__title}>Личный кабинет</h1>
            <p className={styles.cabinet__text}>
                Здравствуйте, {user.name}
            </p>
            <p className={styles.cabinet__email}>{user.email}</p>
            <button
                type="button"
                className={styles.cabinet__logout}
                onClick={logout}
            >
                Выйти
            </button>
        </main>
    );
};

export default CabinetView;
```